

Sliding

Sliding Window Sum

```
#include

using namespace std;
typedef long long ll;
const int maxN = 1e7+1;

int N, K;
ll a, b, c, pre[maxN];

int main(){
    scanf("%d %d", &N, &K);
    scanf("%lld %lld %lld %lld", &pre[1], &a, &b, &c);

    for(int i = 2; i <= N; i++)
        pre[i] = (a * pre[i-1] + b) % c;

    for(int i = 2; i <= N; i++)
        pre[i] += pre[i-1];

    ll xum = 0;
    for(int i = K; i <= N; i++)\
        xum ^= (pre[i] - pre[i-K]);
    printf("%lld\n", xum);
}
```

Sliding Window Minimum

```
#include

using namespace std;
typedef long long ll;
typedef pair pii;

struct Window {
    deque dq;
    int l, r;

    Window(){
        l = r = 0;
    }

    void push_back(ll x){
        while(!dq.empty() && dq.back().first >= x)    dq.pop_back();
        dq.push_back({x, r});
        r++;
    }

    void pop_front(){
        if(dq.front().second == l)    dq.pop_front();
        l++;
    }
}
```

```

    }

    ll get_min(){
        return dq.empty() ? -1 : dq.front().first;
    }
};

int N, K;
ll x, a, b, c;

int main(){
    scanf("%d %d", &N, &K);
    scanf("%lld %lld %lld %lld", &x, &a, &b, &c);

    Window window;
    for(int i = 0; i < K; i++){
        window.push_back(x);
        x = (a * x + b) % c;
    }

    ll xum = window.get_min();
    for(int i = K; i < N; i++){
        window.push_back(x);
        window.pop_front();
        xum ^= window.get_min();
        x = (a * x + b) % c;
    }

    printf("%lld\n", xum);
}

```

Sliding Window Xor

```

#include

using namespace std;
typedef long long ll;
const int maxN = 1e7+1;

int N, K;
ll x, a, b, c;

int main(){
    scanf("%d %d", &N, &K);
    scanf("%lld %lld %lld %lld", &x, &a, &b, &c);

    ll xum = 0;
    for(int i = 1; i <= N; i++){
        int l = max(1, i-K+1);
        int r = min(i, N-K+1);
        if((r - l) % 2 == 0)
            xum ^= x;

        x = (a * x + b) % c;
    }
    printf("%lld\n", xum);
}

```

Sliding Window Or

```
#include

using namespace std;
typedef long long ll;
const int maxN = 1e7+5;

int N, K, val[maxN], pre[maxN], suf[maxN];
ll x, a, b, c;

int main(){
    scanf("%d %d", &N, &K);
    scanf("%lld %lld %lld %lld", &x, &a, &b, &c);
    for(int i = 0; i < N; i++){
        val[i] = x;
        x = (a * x + b) % c;
    }

    for(int i = 0; i < N; i++)
        pre[i] = (i % K == 0) ? val[i] : (pre[i-1] | val[i]);
    for(int i = N-1; i >= 0; i--)
        suf[i] = (i % K == K-1) ? val[i] : (suf[i+1] | val[i]);

    int xor_sum = 0;
    for(int i = 0; i <= N-K; i++){
        int l = i, r = i + K - 1;
        int or_val = suf[l] | pre[r];
        xor_sum ^= or_val;
    }

    printf("%d\n", xor_sum);
}
```

Sliding Window Distinct Values

```
#include

using namespace std;
const int maxN = 2e5;

int N, K, x[maxN];
map freq;

int main(){
    scanf("%d %d", &N, &K);
    for(int i = 0; i < N; i++)
        scanf("%d", &x[i]);

    int distinct = 0;
    for(int i = 0; i < K; i++){
        if(freq[x[i]] == 0)
            distinct++;
        freq[x[i]]++;
    }
    printf("%d%c", distinct, (" \n")[N==K]);

    for(int i = K; i < N; i++){
        if(freq[x[i-K]] == 1)
            distinct--;
        freq[x[i-K]]--;
        if(freq[x[i]] == 0)
            distinct++;
        freq[x[i]]++;
    }
}
```

```

        distinct--;
        freq[x[i-K]]--;

        if(freq[x[i]] == 0)
            distinct++;
        freq[x[i]]++;

        printf("%d%c", distinct, (" \n")[i==N-1]);
    }
}

```

Sliding Window Mode

```

#include

using namespace std;
const int maxN = 2e5;

int N, K, best_freq, best_val, a[maxN];
unordered_map freqs;
unordered_map> vals;

int main(){
    scanf("%d %d", &N, &K);
    for(int i = 0; i < N; i++)
        scanf("%d", &a[i]);

    for(int i = 0; i < K; i++)
        freqs[a[i]]++;

    for(int i = 0; i < K; i++){
        vals[freqs[a[i]]].insert(a[i]);
        if(freqs[a[i]] > best_freq || (freqs[a[i]] == best_freq && a[i] < best_val)){
            best_freq = freqs[a[i]];
            best_val = a[i];
        }
    }

    printf("%d%c", best_val, (" \n")[N==K]);
    for(int i = K; i < N; i++){
        vals[freqs[a[i-K]]].erase(a[i-K]);
        vals[--freqs[a[i-K]]].insert(a[i-K]);
        if(best_val == a[i-K]){
            for(int f = freqs[a[i-K]]+1; f > 0; f--){
                if(!vals[f].empty()){
                    best_freq = f;
                    best_val = *vals[f].begin();
                    break;
                }
            }
        }
    }

    vals[freqs[a[i]]].erase(a[i]);
    vals[++freqs[a[i]]].insert(a[i]);
    if(freqs[a[i]] > best_freq || (freqs[a[i]] == best_freq && a[i] < best_val)){
        best_freq = freqs[a[i]];
        best_val = a[i];
    }

    printf("%d%c", best_val, (" \n")[i==N]);
}

```

```
    }
}
```

Sliding Window Mex

```
#include

using namespace std;
const int maxN = 2e5;

int N, K, a[maxN];
set mex_set;
unordered_map freq;

int main(){
    scanf("%d %d", &N, &K);
    for(int i = 0; i < N; i++)
        scanf("%d", &a[i]);

    for(int i = 0; i <= N; i++)
        mex_set.insert(i);

    for(int i = 0; i < K; i++){
        freq[a[i]]++;
        if(freq[a[i]] == 1)
            mex_set.erase(a[i]);
    }

    printf("%d%c", *mex_set.begin(), (" \n")[N==K]);
    for(int i = K; i < N; i++) {
        freq[a[i-K]]--;
        if(freq[a[i-K]] == 0)
            mex_set.insert(a[i-K]);

        freq[a[i]]++;
        if(freq[a[i]] == 1)
            mex_set.erase(a[i]);

        printf("%d%c", *mex_set.begin(), (" \n")[i==N-1]);
    }
}
```

Sliding Window Median

```
#include
#include
#include

using namespace std;
using namespace __gnu_pbds;
typedef pair pii;
const int maxN = 2e5;
```

```

int N, K, x[maxN];
tree, rb_tree_tag,
tree_order_statistics_node_update> T;

int main(){
    scanf("%d %d", &N, &K);
    for(int i = 0; i < K; i++){
        scanf("%d", &x[i]);
        T.insert({x[i], i});
    }

    printf("%d%c", (*T.find_by_order((K-1)/2)).first, (" \n")[N==K]);
    for(int i = K; i < N; i++){
        scanf("%d", &x[i]);
        T.insert({x[i], i});
        T.erase({x[i-K], i-K});
        printf("%d%c", (*T.find_by_order((K-1)/2)).first, (" \n")[i==N-1]);
    }
}

```

Sliding Window Cost

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+1;

int N, K, x[maxN];
ll losum, hisum;
multiset lo, hi;

ll cost(){
    ll median = *lo.rbegin();
    ll locost = (median * lo.size()) - losum;
    ll hicost = hisum - (median * hi.size());
    return locost + hicost;
}

void lowToHigh(){
    ll val = *lo.rbegin();
    lo.erase(lo.lower_bound(val));
    hi.insert(val);
    losum -= val;
    hisum += val;
}

void highToLow(){
    ll val = *hi.begin();
    lo.insert(val);
    hi.erase(hi.lower_bound(val));
    losum += val;
    hisum -= val;
}

void adjust(){
    int tot = (int) (lo.size() + hi.size());
    if(lo.size() < (tot+1)/2) highToLow();
    else if(lo.size() > (tot+1)/2) lowToHigh();
}

```

```

void erase(ll val){
    ll median = *lo.rbegin();
    if(val > median){
        hi.erase(hi.lower_bound(val));
        hisum -= val;
    } else {
        lo.erase(lo.lower_bound(val));
        losum -= val;
    }
    adjust();
}

void insert(ll val){
    if(lo.size() == 0){
        lo.insert(val);
        losum += val;
        return;
    }

    ll median = *lo.rbegin();
    if(val > median){
        hi.insert(val);
        hisum += val;
    } else {
        lo.insert(val);
        losum += val;
    }
    adjust();
}

int main(){
    scanf("%d %d", &N, &K);
    for(int i = 0; i < K; i++){
        scanf("%d", &x[i]);
        insert(x[i]);
    }

    printf("%lld%c", cost(), (" \n")[K==N]);
    for(int i = K; i < N; i++){
        scanf("%d", &x[i]);
        insert(x[i]);
        erase(x[i-K]);
        printf("%lld%c", cost(), (" \n")[i==N-1]);
    }
}

```

Sliding Window Inversions

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+5;

int N, K, a[maxN], ds[maxN];

void update(int idx, int val){
    for(int i = idx; i <= N; i += -i&i)
        ds[i] += val;
}

```

```
int query(int idx){
    int sum = 0;
    for(int i = idx; i > 0; i -= -i&i)
        sum += ds[i];
    return sum;
}

void compress_vals(){
    vector vals;
    for(int i = 0; i < N; i++)
        vals.push_back(a[i]);
    sort(vals.begin(), vals.end());

    unordered_map mp;
    mp.reserve(N);
    for(int i = 0; i < N; i++)
        if(mp.find(vals[i]) == mp.end())
            mp[vals[i]] = i+1;

    for(int i = 0; i < N; i++)
        a[i] = mp[a[i]];
}

int main(){
    scanf("%d %d", &N, &K);
    for(int i = 0; i < N; i++)
        scanf("%d", &a[i]);
    compress_vals();

    ll inversions = 0;
    for(int i = 0; i < K; i++){
        inversions += i - query(a[i]);
        update(a[i], 1);
    }

    printf("%lld%c", inversions, (" \n")[K==N]);
    for(int i = K; i < N; i++){
        inversions -= query(a[i-K]-1);
        update(a[i-K], -1);

        inversions += (K-1) - query(a[i]);
        update(a[i], 1);

        printf("%lld%c", inversions, (" \n")[i==N-1]);
    }
}
```
